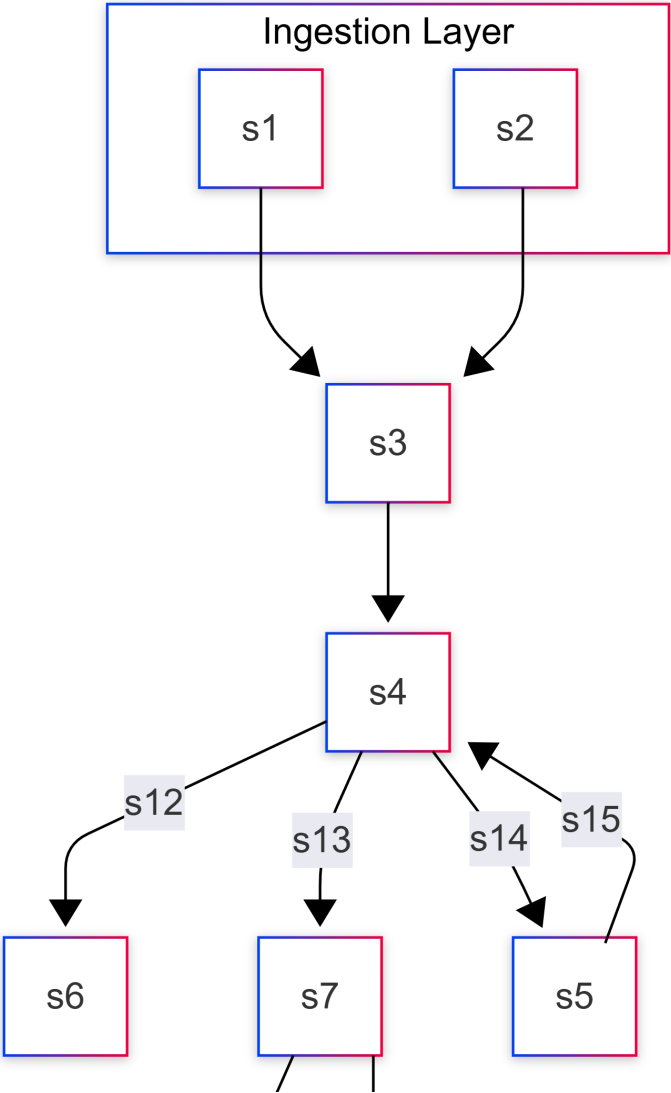


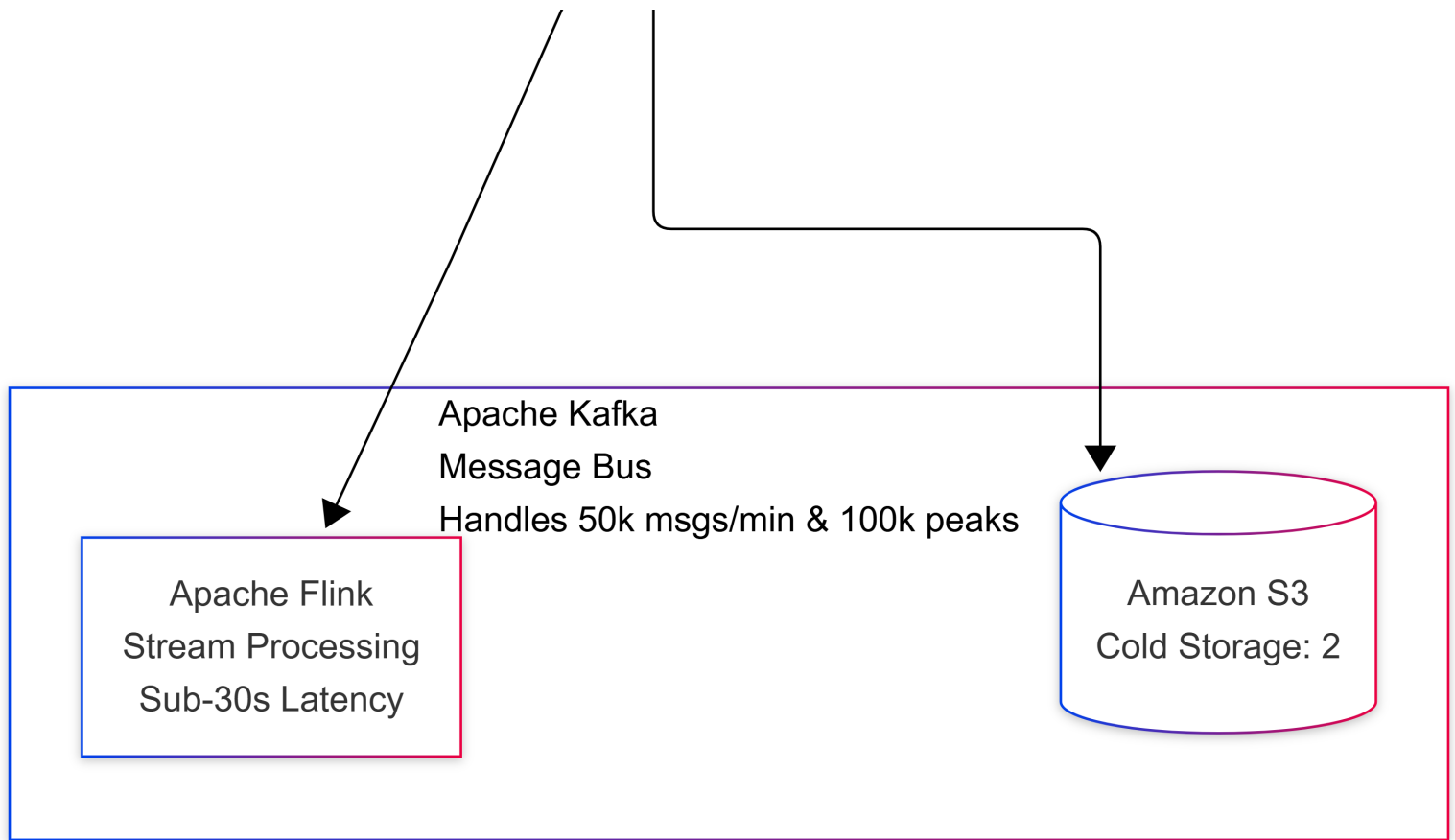
Pulse-Scale: Big Data Engineering Architecture Design

SE446 Architecture Challenge

Deliverable 1: Architecture Diagram

!





Layer Overview:

- **Ingestion:** 2,000 RSS feeds, 5 streaming APIs, and a social-media firehose (peak 100k msgs/sec) all feed into Apache Kafka.
 - **Messaging:** Apache Kafka acts as the central message bus, decoupling ingestion from all downstream systems and absorbing traffic spikes.
 - **Stream Processing:** Apache Flink consumes from Kafka, enforcing sub-30-second end-to-end latency with exactly-once semantics.
 - **Storage:** Flink writes to Amazon S3 (cold, 2 PB, 5-year history) and ClickHouse (hot, 50 TB, last 30 days).
 - **LLM Pipeline:** Flink batches articles per topic into 5-minute windows and sends them to the LLM API. Summaries are written back to ClickHouse.
 - **Analytics:** 500 concurrent analysts run ad-hoc SQL directly against ClickHouse.
 - **Dashboard:** Grafana connects to ClickHouse for live sub-30-second event dashboards and LLM topic summaries.
-

Deliverable 2: Technology Selection Table

Layer	Chosen Tool	Alternatives Considered	Justification (tied to requirements)
Ingestion / Message Bus	Apache Kafka	Amazon SQS, RabbitMQ	Kafka handles 100,000 msgs/sec peak and 50,000 articles/min without bottlenecking. SQS lacks high-throughput pub/sub for multiple consumers at this scale. RabbitMQ cannot sustain 100k+ TPS reliably.
Stream Processing	Apache Flink	Apache Spark Streaming	Flink delivers true event-driven streaming with exactly-once semantics, meeting the sub-30-second latency requirement. Spark Streaming uses micro-batching, adding latency that risks breaching the 30s threshold under heavy load.
Hot Storage	ClickHouse	PostgreSQL, Elasticsearch	ClickHouse is a columnar OLAP database built for lightning-fast ad-hoc SQL on massive datasets, supporting 500 concurrent analysts on 50 TB. PostgreSQL cannot handle 50 TB of analytical queries at this concurrency level. Elasticsearch is ill-suited for complex SQL joins and too expensive at this scale.
Cold Storage	Amazon S3 (Glacier)	HDFS	S3 Glacier tiers cost ~\$0.004/GB/month, making 2 PB financially viable within the \$40k budget. A self-managed HDFS cluster of equivalent size would consume the entire budget in compute and disk alone.
Analytics / Dashboard	ClickHouse + Grafana	Tableau, Apache Superset	Grafana integrates natively with ClickHouse for sub-30-second live dashboards. Tableau is prohibitively expensive at 500-user scale and not optimized for live streaming metrics.

Deliverable 3: Data Partitioning and Storage Strategy

Cold Storage — Amazon S3

- Format:** Apache Parquet with Snappy compression. Columnar format minimizes scan costs for historical queries.
- Partition Key:** `year=YYYY/month=MM/day=DD/source_type=TYPE/` — aligns with time-bounded historical access patterns over a 5-year window.

- **File Size Policy:** 128–256 MB target file sizes using Flink's `StreamingFileSink` rolling policies, avoiding the small-files problem which degrades S3 read throughput.

Hot Storage — ClickHouse

- **Engine:** `MergeTree` — ClickHouse's primary engine, which physically sorts data on disk according to the sort key.
- **Sort / Primary Key:** `(topic, timestamp, source_id)` — enables instant range scans by topic and time window, directly serving the most common analyst and dashboard queries.
- **Secondary Indexes:** Bloom filter indexes on `author_id` and `keywords` for fast point lookups without full column scans.
- **Schema Design:** Fully denormalized wide table. No JOINS at query time — article content, source metadata, sentiment scores, and LLM summaries coexist in one row to maximize read speed for the live dashboard.

Example query performance justification: A query like *"Count articles about 'Politics' from CNN in the last 7 days"* uses the `(topic, timestamp)` sort key to binary-search directly to the relevant data range, skipping all unrelated columns. This returns results in milliseconds rather than minutes against 50 TB.

Deliverable 4: Scalability, Fault Tolerance, and Cost Optimization Strategy

Scalability

- **Kafka** scales horizontally by adding partitions and brokers. Partition count is set to at least 3× the peak consumer parallelism.
- **Flink** scales by adding TaskManagers (via Kubernetes autoscaling). If Kafka consumer lag grows, Flink auto-scales to burn through the backlog.
- **ClickHouse** scales by adding read replicas for analyst query load and sharding by `topic` across nodes.
- **Firehose Skew fix (Problem a):** Kafka partition key is salted — `hash(account_id) + random(1, N)` — to distribute hot accounts evenly. Flink uses two-phase aggregation: first locally aggregate per salt key, then merge into a final per-account result.

Fault Tolerance

- Kafka is deployed with **replication factor 3** across 3 availability zones (AZs). If a broker dies, the Kafka controller elects a new partition leader within seconds.
- Flink checkpoints state to S3 every 30 seconds. If a Flink node fails, it restarts from the last checkpoint with no data loss.
- ClickHouse data is replicated across AZs using `ReplicatedMergeTree`.
- If the LLM API rate-limits, Flink queues prompts in a Dead Letter Topic in Kafka and replays them once the limit lifts. The dashboard degrades gracefully with a "Summary temporarily unavailable" message.

Cost Optimization

- **LLM Bill reduction (Problem c):** Articles per topic are batched into a single 5-minute prompt (one API call per batch instead of one per article), deduplicated, and stripped of HTML boilerplate before sending. A Redis semantic cache avoids re-summarizing near-identical article clusters.
- **Dashboard latency fix (Problem e):** ClickHouse Materialized Views pre-aggregate the top dashboard metrics (e.g., top topics per hour) incrementally. Dashboard queries hit the pre-aggregated table, not the 50 TB raw table, dropping p95 latency from 12s to under 2s.

Monthly Cost Breakdown:

Component	Estimated Cost
LLM API	\$18,000
ClickHouse (50 TB EBS + 4 compute nodes)	\$8,800
Amazon S3 Glacier (2 PB cold)	\$10,000
Kafka + Flink compute clusters	\$3,000
Total	\$39,800 / month

This comes in under the **\$40,000/month** hard cap by using Glacier-tier cold storage for the 2 PB historical archive.

Deliverable 5: Operational Plan

Monitoring

- **Infrastructure metrics:** Kafka consumer lag, Flink checkpoint duration, ClickHouse query p95/p99 latency, and S3 write throughput are all tracked via **Prometheus + Grafana** dashboards.
- **Alerting:** PagerDuty alerts fire if: Kafka consumer lag exceeds 5 minutes of data, Flink checkpoint fails 3 times consecutively, ClickHouse p95 query latency exceeds 5 seconds, or LLM API error rate exceeds 10%.
- **LLM quality monitoring:** A random sample of 5% of generated summaries are logged and spot-checked weekly for hallucinations or quality degradation.
- **Budget monitoring:** AWS Cost Explorer alerts trigger at 80% and 95% of the monthly \$40,000 cap.

Failure Recovery

Failure Scenario	Impact	Recovery
Kafka broker dies	Momentary producer/consumer timeout	Kafka controller elects new partition leader within seconds; clients auto-reconnect
AZ goes down	Partial compute loss	Flink restarts from last S3 checkpoint in surviving AZs; Kafka serves data from replicas
Ingestion spike (3× normal)	Dashboard latency may spike	Flink auto-scales via Kubernetes; Kafka absorbs the burst; system catches up
LLM API rate-limited	Summaries pause	Prompts queued in Kafka DLQ; replayed after limit lifts; UI shows graceful degradation

Deployment

- **Infrastructure as Code:** All infrastructure (Kafka, Flink, ClickHouse, S3 buckets) is provisioned via **Terraform**, enabling reproducible deployments and disaster recovery in a second region within 30 minutes.
- **CI/CD:** Flink job updates are deployed via a **rolling restart** strategy — new job versions are started and old ones drained gracefully, ensuring zero downtime.
- **Team of 4 scope:** One engineer owns Kafka/Flink pipeline, one owns ClickHouse and analytics, one owns the LLM integration and cost controls, and one handles infrastructure/monitoring. On-call rotates weekly.